

**ĐẠI HỌC QUỐC GIA HÀ NỘI**  
**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**



**BÀI TẬP TOÁN RỜI RẠC VÀ THUẬT TOÁN**

**Nhóm 4**

Nguyễn Thị Nhật Linh - 25007871

Lê Thị Liên - 25007874

Trần Tùng Lâm - 25007944

Nguyễn Phúc Lợi - 25007868

# Mục lục

|                               |           |
|-------------------------------|-----------|
| <b>Bài 1: Thuật toán.....</b> | <b>3</b>  |
| <b>Bài 2: Ứng dụng.....</b>   | <b>12</b> |

## Bài tập 3: Một số thuật toán trên đồ thị và ứng dụng

### Bài 1: Thuật toán

Hãy trình bày những nội dung về ít nhất 02 thuật toán trên đồ thị: khái niệm, các yếu tố liên quan, ý tưởng và lược đồ thuật toán, định hướng ứng dụng.

Chọn 1 trong 3 thuật toán sau và một thuật toán tùy chọn khác để thực hiện:

- Chu trình/đường Hamilton.
- Chu trình/đường Euler.
- Tô màu đồ thị.

#### 1) Chu trình và đường đi Hamilton

##### 1.1) Định nghĩa

Cho đồ thị  $G=(V,E)$  có  $n$  đỉnh. Ta có:

- Một chu trình  $x_1, x_2, \dots, x_n$  được gọi là chu trình Hamilton nếu là đường Hamilton **bắt đầu và kết thúc tại cùng một đỉnh**.
- Đường đi  $x_1, x_2, \dots, x_n$  được gọi là đường đi Hamilton đường đi trong đồ thị đi qua **mỗi đỉnh đúng một lần**..

##### 1.2) Định lí

###### a) Định lí 1 (Điều kiện cần)

Cho đồ thị vô hướng  $G=(V,E)$  nếu tồn tại tập đỉnh  $S \subseteq V$ ,  $|S| = k$  sao cho khi loại bỏ tập  $S$  và các cạnh liên thuộc của nó, đồ thị còn lại  $G - S$  có số thành phần liên thông  $c(G-S) > k$  thì  $G$  không phải đồ thị Hamilton

###### b) Định lí Dirak 1952

Xét đơn đồ thị vô hướng  $G = (V, E)$  có  $n$  đỉnh ( $n \geq 3$ ). Nếu mọi

đỉnh đều có bậc lớn hơn hoặc bằng  $\left\lfloor \frac{n}{2} \right\rfloor$  thì  $G$  là đồ thị Hamilton

c) Định lí Ghouila – Houiri 1960

Xét đơn đồ thị có hướng liên thông mạnh  $G = (V, E)$  có  $n$  đỉnh. Nếu trên phiên bản vô hướng của  $G$ , mọi đỉnh đều có bậc không nhỏ hơn  $n$  thì  $G$  là đồ thị Hamilton

d) Định lí Ore 1960

Nếu với mọi cặp đỉnh  $u, v$  không kề nhau thì  $\deg(u) + \deg(v) \geq n$  thì  $G$  là đồ thị Hamilton

e) Định lí Meynie 1973

*Xét đơn đồ thị có hướng liên thông mạnh  $G=(V, E)$  có  $n$  đỉnh. Nếu trên phiên bản vô hướng của  $G$  mọi cặp đỉnh không kề nhau đều có tổng bậc lớn hơn hoặc bằng  $2n-1$  thì  $G$  là đồ thị Hamilton.*

f) Định lí Bondy 1972

*Xét đồ thị vô hướng  $G=(V, E)$  có  $n$  đỉnh, với mỗi cặp đỉnh không kề nhau  $(u, v)$  mà  $\deg(u) + \deg(v) \geq n$ , ta thêm một cạnh nối  $u$  và  $v$ . Cứ làm như vậy tới khi không thêm được cạnh nào nữa, thì ta thu được một bao đóng của đồ thị  $G$ , kí hiệu  $cl(G)$ . Khi đó  $G$  là đồ thị Hamilton nếu và chỉ nếu  $cl(G)$  là đồ thị Hamilton*

### 1.3) Ý tưởng thuật toán và mã giả

Thuật toán tìm chu trình Hamilton thường sử dụng phương pháp **quay lui** để xây dựng dần đường đi

a) Ý tưởng

1. Chọn một đỉnh bất kỳ làm điểm bắt đầu.
2. Tại mỗi bước, chọn một đỉnh kề với đỉnh hiện tại và **chưa được thăm** để tiếp tục mở rộng đường đi.
3. Đánh dấu đỉnh vừa đi qua để tránh lặp lại.

4. Nếu đã đi qua tất cả các đỉnh:
    - o Kiểm tra xem đỉnh cuối có nối lại với đỉnh bắt đầu hay không.
    - o Nếu có thì thu được một chu trình Hamilton.
  5. Nếu không còn đỉnh hợp lệ để đi tiếp:
    - o Quay lui về bước trước đó và thử lựa chọn khác.
  6. Lặp lại quá trình cho đến khi tìm được chu trình hoặc đã thử hết mọi khả năng.
- b) Mã giả

```

HAMILTON(G):
    Chọn đỉnh bắt đầu s
    path[0] ← s
    Đánh dấu s đã thăm

    Nếu BACKTRACK(1) = TRUE:
        Xuất path (chu trình Hamilton)
    Ngược lại:
        Thông báo không tồn tại chu trình Hamilton

BACKTRACK(pos):
    Nếu pos = n:
        Nếu tồn tại cạnh nối path[n-1] với path[0]:
            Trả về TRUE
        Ngược lại:
            Trả về FALSE

    với mỗi đỉnh v kề với path[pos-1]:
        Nếu v chưa được thăm:
            path[pos] ← v
            Đánh dấu v đã thăm

            Nếu BACKTRACK(pos + 1) = TRUE:
                Trả về TRUE

        Bỏ đánh dấu v (quay lui)

    Trả về FALSE
  
```

#### 1.4) Đánh giá và ứng dụng thực tiễn

Độ phức tạp thuật toán trong trường hợp xấu nhất là  $O(n!)$

Thuật toán được áp dụng trong một số bài toán thực tế như:

1. Bài toán người du lịch (Travelling Salesman Problem – TSP)

Chu trình Hamilton là nền tảng của bài toán người du lịch, trong đó người bán hàng cần đi qua tất cả các thành phố đúng một lần và quay trở về điểm xuất phát sao cho tổng quãng đường là ngắn nhất. Bài toán này được ứng dụng rộng rãi trong logistics, giao hàng và lập kế hoạch vận chuyển.

## 2. Lập kế hoạch lộ trình và định tuyến

Chu trình Hamilton được sử dụng để xây dựng lộ trình đi qua tất cả các điểm cần kiểm tra hoặc giao hàng mà không lặp lại vị trí. Điều này giúp tối ưu hóa việc di chuyển của xe vận tải, máy bay không người lái (drone) và các hệ thống tự động.

## 3. Thiết kế vi mạch và bo mạch điện tử

Trong thiết kế mạch tích hợp, chu trình Hamilton được áp dụng để sắp xếp vị trí các linh kiện và đường dẫn tín hiệu sao cho tín hiệu đi qua tất cả các điểm cần thiết mà không lặp lại, giúp giảm độ trễ và tiết kiệm không gian.

## 4. Sinh học và hóa học tính toán

Chu trình Hamilton được sử dụng để mô hình hóa cấu trúc phân tử, phân tích trình tự ADN và giải quyết các bài toán liên quan đến việc sắp xếp các thành phần sinh học theo thứ tự tối ưu.

## 5. Lập lịch và tối ưu hóa tài nguyên

Trong các hệ thống sản xuất và phân công công việc, chu trình Hamilton giúp xây dựng thứ tự thực hiện các nhiệm vụ sao cho mỗi công việc chỉ được thực hiện một lần, tránh trùng lặp và giảm chi phí vận hành.

## 6. Trí tuệ nhân tạo và robot

Trong robot tự hành và trí tuệ nhân tạo, chu trình Hamilton được sử dụng để lập kế hoạch đường đi qua tất cả các điểm quan trọng trong môi trường mà không quay lại vị trí đã đi qua, giúp tối ưu thời gian và năng lượng.

## 2) Chu trình Euler

### 2.1) Định nghĩa

Đường Euler trong một đồ thị là một đường đi sao cho **mỗi cạnh của đồ thị được đi qua đúng một lần và không lặp lại cạnh nào**.

Chu trình Euler là một trường hợp đặc biệt của đường Euler. Là đường đi thỏa mãn điều kiện đi qua **tất cả các cạnh đúng một lần**, và **quay trở lại đúng đỉnh xuất phát ban đầu**.

## 2.2) Định lý

### a) Định lý 1 (Đồ thị vô hướng)

Một đồ thị vô hướng liên thông  $G = (V, E)$  có **chu trình Euler** khi và chỉ khi **mọi đỉnh của đồ thị đều có bậc chẵn**, tức là:

Mỗi đỉnh trong đồ thị đều có số cạnh kề với nó là một số chẵn.

Chúng minh

Giả sử đồ thị  $G$  có một chu trình Euler. Khi đi dọc theo chu trình này, mỗi lần ta đi qua một đỉnh thì ta luôn đi vào bằng một cạnh và đi ra bằng một cạnh khác. Do đó, mỗi lần xuất hiện tại một đỉnh, bậc của đỉnh đó tăng thêm hai đơn vị. Vì chu trình Euler đi qua tất cả các cạnh của đồ thị nên suy ra mọi đỉnh đều có bậc chẵn.

Ngược lại, nếu đồ thị  $G$  là liên thông và tất cả các đỉnh đều có bậc chẵn thì ta có thể xây dựng được một thuật toán để lần lượt đi qua toàn bộ các cạnh đúng một lần và quay trở lại điểm xuất phát. Điều này chứng tỏ đồ thị tồn tại một chu trình Euler.

### Hệ quả 1 (Đồ thị vô hướng)

Một đồ thị vô hướng liên thông  $G=(V, E)$  có **đường đi Euler** khi và chỉ khi nó **có đúng hai đỉnh bậc lẻ**, còn tất cả các đỉnh còn lại đều có bậc chẵn.

Chúng minh

Nếu đồ thị có một đường đi Euler thì chỉ có hai đỉnh đặc biệt là đỉnh bắt đầu và đỉnh kết thúc của đường đi có bậc lẻ. Tất cả các đỉnh còn lại đều có bậc chẵn, vì tại mỗi đỉnh này ta luôn đi vào và đi ra đúng một lần.

Ngược lại, nếu đồ thị liên thông và có đúng hai đỉnh bậc lẻ, ta có thể thêm tạm thời một cạnh nối hai đỉnh này. Khi đó, đồ thị mới sẽ có tất cả các đỉnh bậc chẵn nên tồn tại chu trình Euler. Sau khi tìm được chu trình Euler, ta loại bỏ cạnh giả vừa thêm vào thì thu được một đường đi Euler của đồ thị ban đầu.

## **b) Định lý 2 (Đồ thị có hướng)**

Một đồ thị có hướng liên thông yếu  $G=(V, E)$  có **chu trình Euler** khi và chỉ khi **tại mọi đỉnh, bán bậc ra bằng bán bậc vào**. Nói cách khác, với mỗi đỉnh của đồ thị, số cạnh đi ra khỏi đỉnh đó phải bằng số cạnh đi vào đỉnh đó.

Chứng minh

Cách chứng minh hoàn toàn tương tự như trường hợp đồ thị vô hướng. Khi đi theo một chu trình Euler có hướng, mỗi lần đến một đỉnh ta phải đi vào bằng một cung và đi ra bằng một cung khác. Do đó, tổng số cung đi vào luôn bằng tổng số cung đi ra tại mỗi đỉnh. Ngược lại, nếu điều kiện này được thỏa mãn và đồ thị liên thông yếu thì ta có thể xây dựng được một chu trình Euler.

## **Hệ quả 2 (Đồ thị có hướng)**

Một đồ thị có hướng liên thông yếu  $G=(V, E)$  có **đường đi Euler nhưng không có chu trình Euler** khi và chỉ khi tồn tại đúng hai đỉnh đặc biệt  $s$  và  $t$  sao cho:

- Tại đỉnh  $s$ : số cạnh đi ra nhiều hơn số cạnh đi vào đúng một đơn vị (đỉnh bắt đầu),
- Tại đỉnh  $t$ : số cạnh đi vào nhiều hơn số cạnh đi ra đúng một đơn vị (đỉnh kết thúc),

còn tất cả các đỉnh khác đều có số cạnh đi vào bằng số cạnh đi ra.

### 2.3) Ý tưởng thuật toán tìm chu trình Euler

#### 2.3.1) thuật toán Hierholzer (1873)

##### a) Ý tưởng

1. Bắt đầu từ một đỉnh bất kỳ.
2. Đi theo các cạnh chưa dùng.



3. Khi bị kẹt: Quay lại điểm có cạnh chưa dùng.
4. Ghép các chu trình con thành chu trình Euler hoàn chỉnh

b) Mã giả

```
HIERHOLZER(G):  
  Chọn đỉnh bắt đầu s  
  Stack ← {s}  
  Path ← rỗng  
  
  While Stack không rỗng:  
    v = đỉnh trên cùng Stack  
    Nếu v còn cạnh chưa dùng:  
      Chọn cạnh (v, u)  
      Xóa cạnh (v, u)  
      Push u vào Stack  
    Ngược lại:  
      Pop v khỏi Stack  
      Thêm v vào Path  
  
  Path chính là chu trình Euler
```

### 2.3.2) thuật toán Fleury

a) ý tưởng

1. Bắt đầu từ một đỉnh bất kỳ của đồ thị Euler.
2. Ở mỗi bước, chọn một cạnh kề để đi tiếp sao cho:
  - + Ưu tiên chọn **cạnh không phải là cầu**.
  - + Chỉ đi qua **cạnh cầu** khi không còn lựa chọn nào khác.
3. Sau khi đi qua một cạnh, ta **loại bỏ cạnh đó khỏi đồ thị**.
4. Lặp lại quá trình cho đến khi không còn cạnh nào để đi.
5. Đường đi thu được chính là **chu trình Euler**.

b) Mã giả

```

FLEURY(G):
  Chọn một đỉnh bắt đầu s
  u ← s

  While còn tồn tại cạnh trong G:
    Với mỗi cạnh (u, v) kề u:
      Nếu (u, v) không phải là cầu
      hoặc u chỉ còn đúng một cạnh kề:
        Chọn cạnh (u, v)
        Break

    Ghi u vào đường đi Euler
    Xóa cạnh (u, v) khỏi G
    u ← v

  Ghi đỉnh cuối cùng vào đường đi
  Trả về đường đi Euler

```

### 2.2.3) So sánh thuật toán Hierholzer và Fleury

Thuật toán Fleury có ưu điểm là dễ hiểu và trực quan, vì ở mỗi bước thuật toán lựa chọn cạnh đi tiếp dựa trên nguyên tắc tránh cạnh cầu. Tuy nhiên, nhược điểm lớn của Fleury là hiệu năng thấp. Ở mỗi bước, thuật toán phải kiểm tra xem một cạnh có phải là cầu hay không, mà việc kiểm tra này có độ phức tạp là  $O(V+E)$ . Do phải thực hiện thao tác này cho nhiều cạnh, nên độ phức tạp tổng thể của Fleury có thể lên tới  $O(E \times (V+E))$ . Vì vậy, Fleury không phù hợp cho các đồ thị lớn.

Ngược lại, thuật toán Hierholzer có ưu điểm vượt trội về tốc độ. Thuật toán này xây dựng các chu trình con và ghép chúng lại với nhau, đồng thời mỗi cạnh trong đồ thị chỉ được duyệt đúng một lần. Do đó, độ phức tạp của Hierholzer chỉ là  $O(E)$ . Nhờ hiệu năng cao, Hierholzer được sử dụng rộng rãi trong các ứng dụng thực tế và các hệ thống xử lý đồ thị lớn. Nhược điểm của thuật toán này là cách cài đặt phức tạp hơn và khó hiểu hơn so với Fleury.

## 2.4) Ứng dụng

### 1. Lập lộ trình giao thông và vận chuyển

Chu trình Euler được dùng để tìm lộ trình đi qua tất cả các tuyến đường đúng một lần và quay về điểm xuất phát. Ứng dụng phổ biến trong việc lập đường đi cho xe thu gom rác, xe quét đường, xe phát thư hoặc xe bảo trì hạ tầng giao thông, giúp tiết kiệm thời gian và nhiên liệu.

### 2. Robot và tự động hóa

Trong robot lau nhà, robot kiểm tra đường ống hoặc robot khảo sát khu vực, chu trình Euler giúp robot đi qua tất cả các lối đi hoặc hành lang đúng một lần mà không bị lặp lại, từ đó tối ưu hóa quãng đường di chuyển.

### 3. Thiết kế mạch điện và mạng dây

Trong thiết kế mạch điện hoặc hệ thống cáp mạng, chu trình Euler được dùng để tối ưu hóa việc đi dây, đảm bảo tất cả các kết nối được kiểm tra hoặc thiết lập mà không cần đi lại nhiều lần, giúp giảm chi phí vật liệu.

### 4. Tin sinh học (Bioinformatics)

Chu trình Euler được sử dụng trong việc lắp ráp chuỗi ADN từ các đoạn nhỏ thông qua đồ thị De Bruijn. Đây là một ứng dụng rất quan trọng trong giải mã gen và nghiên cứu sinh học phân tử.

### 5. Bài toán lập lịch và phân công công việc

Chu trình Euler được áp dụng trong các bài toán lập lịch, nơi mỗi nhiệm vụ hoặc tuyến công việc cần được thực hiện đúng một lần, giúp tối ưu hóa thứ tự thực hiện và tránh trùng lặp.

### 6. Trò chơi và bài toán giải đố

Nhiều trò chơi nổi tiếng, vẽ hình bằng một nét hoặc bài toán mê cung sử dụng nguyên lý chu trình Euler để xác định xem có thể vẽ hình mà không nhấc bút hay không.

## **Bài 2: Ứng dụng**

Trình bày và triển khai một ứng dụng của một thuật toán thao tác trên đồ thị đã thực hiện trong bài 1 vào một bài toán cụ thể có tính thực tiễn.

- Phát biểu bài toán.
- Áp dụng thuật toán.
- Cài đặt chương trình.

**Bài toán:** Giả sử một trường đại học cần xếp lịch thi cho  $n$  môn học. Mục tiêu là xếp các môn vào các ca thi sao cho có các ràng buộc sau đây

Ràng buộc cứng:

- Một ngày có 2 ca thi
- Một ca thi có tối đa 2 môn thi
- Một sinh viên không thể thi hai môn cùng lúc

Ràng buộc mềm:

- Ưu tiên xếp các môn thi có đông người trùng thi trước

**Chuyển đổi sang đồ thị:**

- **Đỉnh (V):** Mỗi môn học là một đỉnh của đồ thị.
- **Cạnh (E):** Nối giữa hai môn thi khi có sinh viên đăng ký học cả hai môn đó (Xung đột)
- **Màu sắc:** Mỗi màu đại diện cho một ca thi (buổi thi) khác nhau.
- **Mục tiêu:** Dùng tô màu đồ thị để sắp xếp được lịch thi cho các môn sao cho thỏa mãn các ràng buộc của đề bài

**INPUT:** Input của bài toán có thể lấy số liệu trong cơ sở dữ liệu để tính toán, tuy nhiên ở trong bài báo cáo này sẽ sử dụng file csv để thuận lợi hơn.

- Bảng danh sách tên sinh viên
- Bảng danh sách môn học
- Bảng danh sách sinh viên đăng ký môn học

**OUTPUT:** Bảng các ca thi đã được sắp xếp

## **Áp dụng thuật toán - Các bước thực hiện bài toán**

**B1:** Đọc file dữ liệu và xử lý

- Đọc các bảng sinh viên, môn học, và sinh viên đăng ký môn học
- Xử lý số liệu đã đọc và tổng hợp lại từng môn học có những sinh viên nào học

**B2:** Xây dựng ma trận kề

- Từ số liệu môn học và sinh viên đã xử lý ở trên, ta tính toán từng môn có bao nhiêu sinh viên đăng ký trùng bằng công thức:

$$P12 = (T12/L1) * 100$$

Trong đó:

P12: Tỷ lệ sinh viên học trùng giữa môn học 1 và 2

T12: Số lượng sinh viên học trùng giữa 2 môn học

L1: Tổng số sinh viên học môn thứ 1

- Sau khi tính toán được tỷ lệ sinh viên đăng ký trùng thì ta có thể tìm được ma trận kề và vẽ được đồ thị

**B3:** Sắp xếp lại các đỉnh của đồ thị

- Vì các môn có càng nhiều xung đột sẽ càng khó để sắp xếp nên ta sẽ sắp xếp lại để có thể ưu tiên xếp các môn này lên các ca thi phía trước.
- Đỉnh nào có càng nhiều bậc thì sẽ được cho lên đầu

**B4:** Áp dụng tô màu đồ thị và chia ca thi

- Sau khi sắp xếp lại các đỉnh theo thứ tự ưu tiên thì ta sẽ áp dụng thuật toán tô màu đồ thị
- Ta sẽ tô màu đỉnh đầu tiên bằng màu thứ 1
- Duyệt các đỉnh còn lại chưa tô màu, nếu ko nối với đỉnh 1 và chưa tô màu thì sẽ được tô bằng màu thứ 1
- Nếu đã có đủ 2 đỉnh được tô màu (ràng buộc cứng đề bài - 1 ca thi có 2 môn thi) thì sẽ break và tiếp tục với ca tiếp theo
- Lặp lại với các màu khác cho đến khi tất cả các đỉnh được tô màu
- Sau khi chạy xong thuật toán thì sẽ cho ra kết quả là các ca thi

### **Mã giả cho thuật toán tô màu đồ thị**

Algorithm: Exam\_Scheduling\_Coloring

Input: Graph  $G = (V, E)$  với  $V$  là danh sách môn học,  $E$  là các xung đột. (Đây là sau bước tìm được ma trận kề và vẽ được đồ thị)

Output: Phân loại môn học vào các ca thi.

1. Sắp xếp  $V$  theo thứ tự bậc giảm dần (degree).
2. Khởi tạo mảng `result_colors[]` lưu màu của từng đỉnh, mặc định = -1.
3. Khởi tạo biến `current_color = 0` là màu hiện tại.
3. Cho mỗi đỉnh  $u$  thuộc  $V$ :
  - a. Tạo một biến `can_color = True`.
  - b. Duyệt các đỉnh  $v$  kề với  $u$ :
 

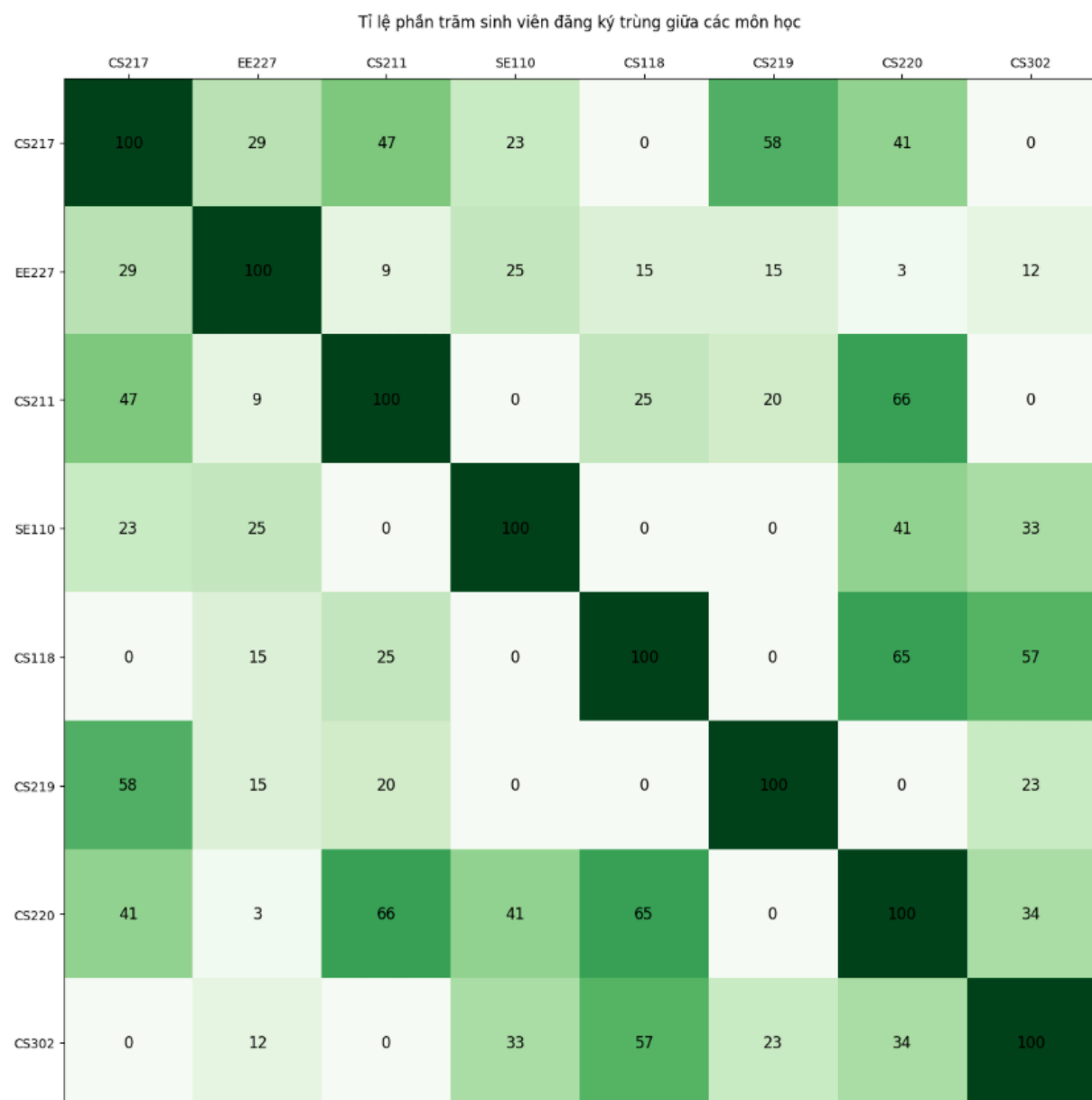
Nếu  $v$  đã được tô màu `current_color` và xung đột với  $u$  thì sẽ bỏ qua.

Nếu không thì đỉnh đó sẽ được tô màu `current_color` và gán vào `result_colors`
  - c. Nếu đã có 2 môn thi được tô màu trong 1 ca thì ta chuyển sang lần quét tiếp theo (Ràng buộc cứng đề bài)
4. Trả về mảng `result_colors`.

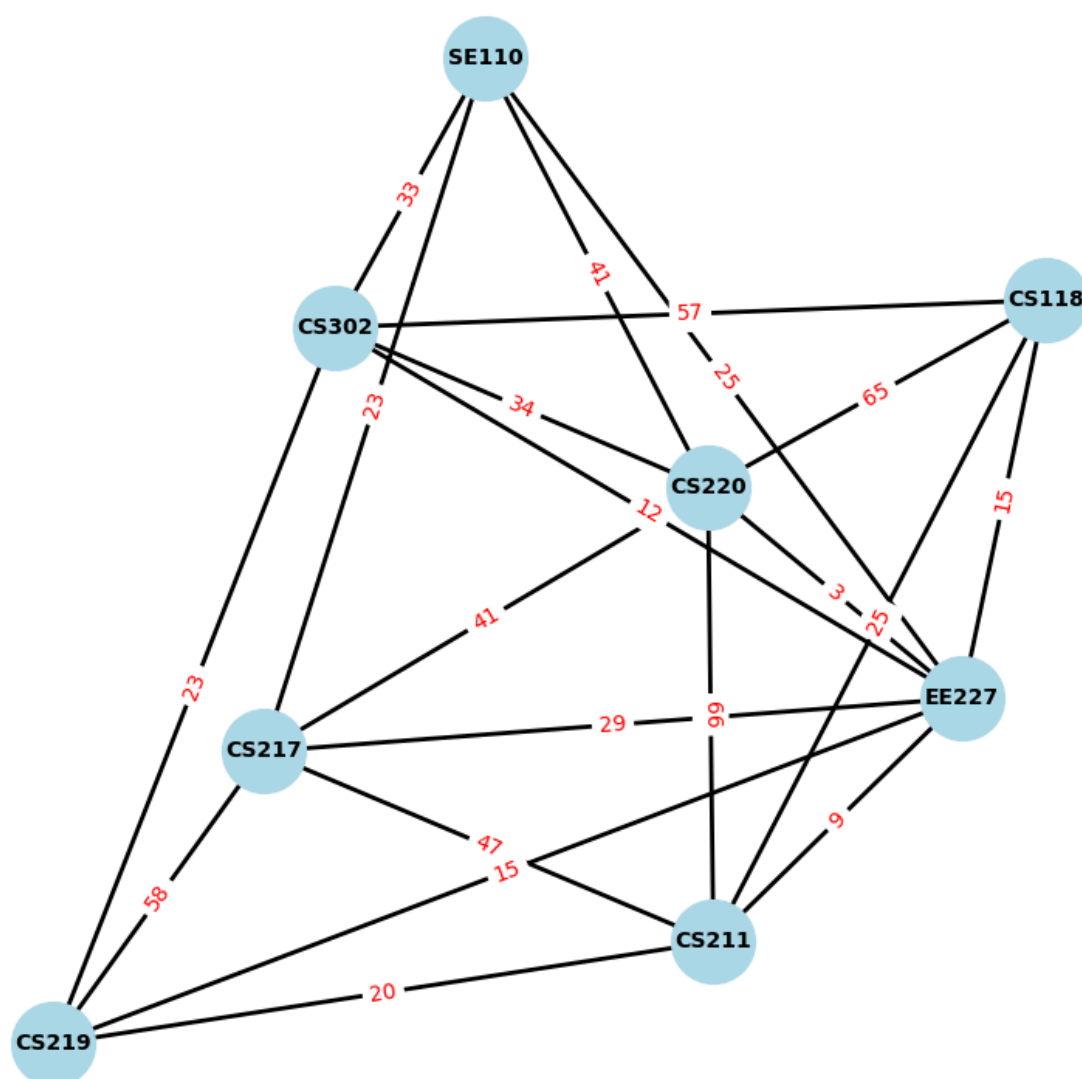
## Cài đặt chương trình

Source code chạy chương trình sẽ nằm trong file **coloring.ipynb**, các file dữ liệu sẽ ở trong thư mục dataset, trong đó bao gồm 3 file csv là các bảng sinh viên, môn học, và sinh viên đăng ký môn học. Những file số liệu này là số liệu thực tế được cắt ra từ nguồn trên internet.

Từ bước đọc số liệu và xử lý, ta có thể xuất ra được ma trận kề và vẽ được đồ thị của bài toán



Ma trận kề thể hiện tỉ lệ trùng giữa các môn học



Bên trên là đồ thị được vẽ ra từ ma trận kề đã được tính ra bên trên, trong đó các đỉnh là mã môn học, còn các cạnh là tỉ lệ sinh viên đăng ký trùng

Từ đồ thị thì ta sắp xếp lại các đỉnh theo thứ tự xung đột nhiều nhất sẽ được xếp lên đầu. Từ đó ta sẽ thu được các môn học được sắp xếp như sau:



```
Môn: EE227, Bậc: 7
Môn: CS220, Bậc: 6
Môn: CS217, Bậc: 5
Môn: CS211, Bậc: 5
Môn: CS302, Bậc: 5
Môn: SE110, Bậc: 4
Môn: CS118, Bậc: 4
Môn: CS219, Bậc: 4
```

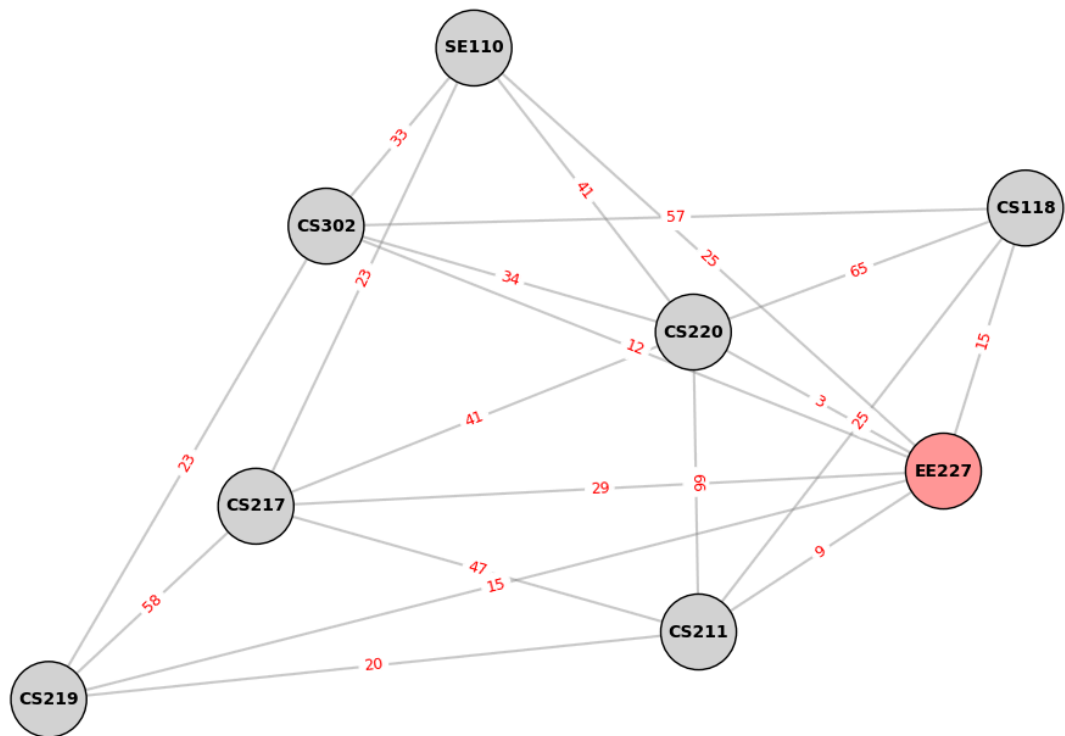
Giờ ta áp dụng thuật toán tô màu đồ thị cho các đỉnh của đồ thị. Thuật toán như sau:

```
# Thuật toán tô màu đồ thị
result_colors = [-1] * n # Lưu ca thi của từng môn
current_color = 0
while -1 in result_colors:
    # Biến này dùng để check 2 môn trong cùng 1 ca thi
    num = 0
    for node_idx, _ in sorted_nodes:
        if result_colors[node_idx] == -1:
            # Kiểm tra xem môn này có xung đột với các môn đã có trong ca hiện tại không
            can_color = True
            for neighbor_idx in range(n):
                # Nếu có trùng (matrix > 0) và môn hàng xóm đó đã được gán ca hiện tại
                if adj_matrix[node_idx][neighbor_idx] > 0 and result_colors[neighbor_idx] == current_color:
                    can_color = False
                    break

            if can_color:
                num += 1
                result_colors[node_idx] = current_color
        if (num == 8):
            break
    current_color += 1
```

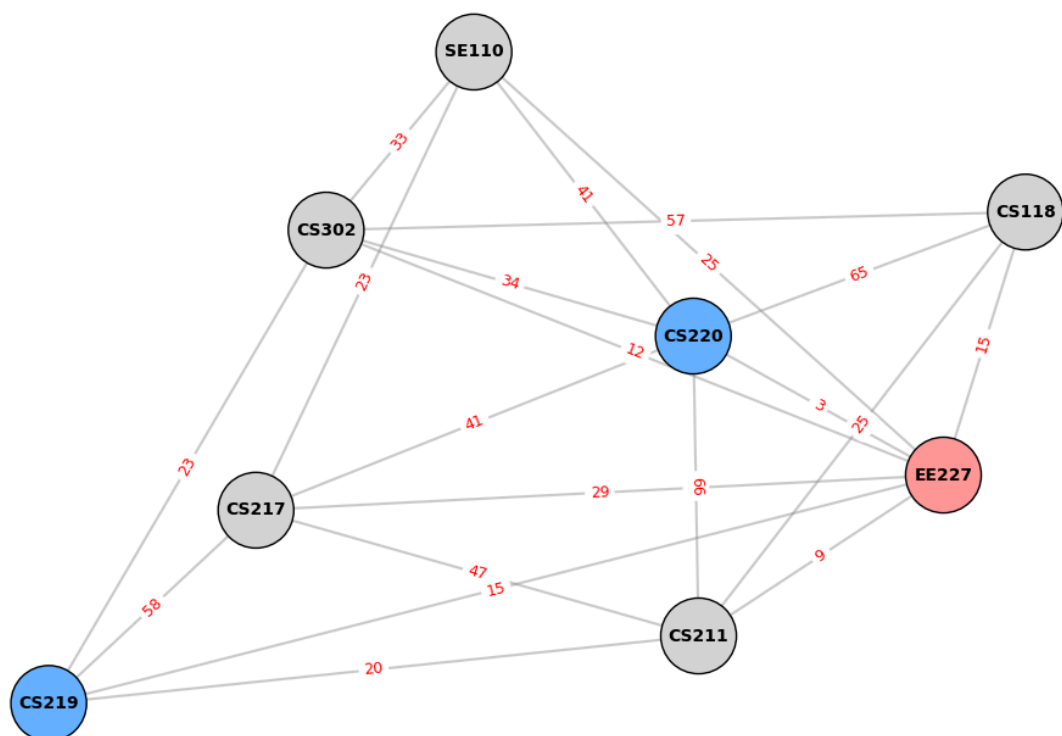
Với vòng quét thứ nhất, ta thu được đỉnh có xung đột cao nhất sẽ là một màu riêng, và không có đỉnh nào có thể xếp chung được, nên môn EE227 sẽ được cho riêng là 1 màu

VÒNG QUÉT CA 1

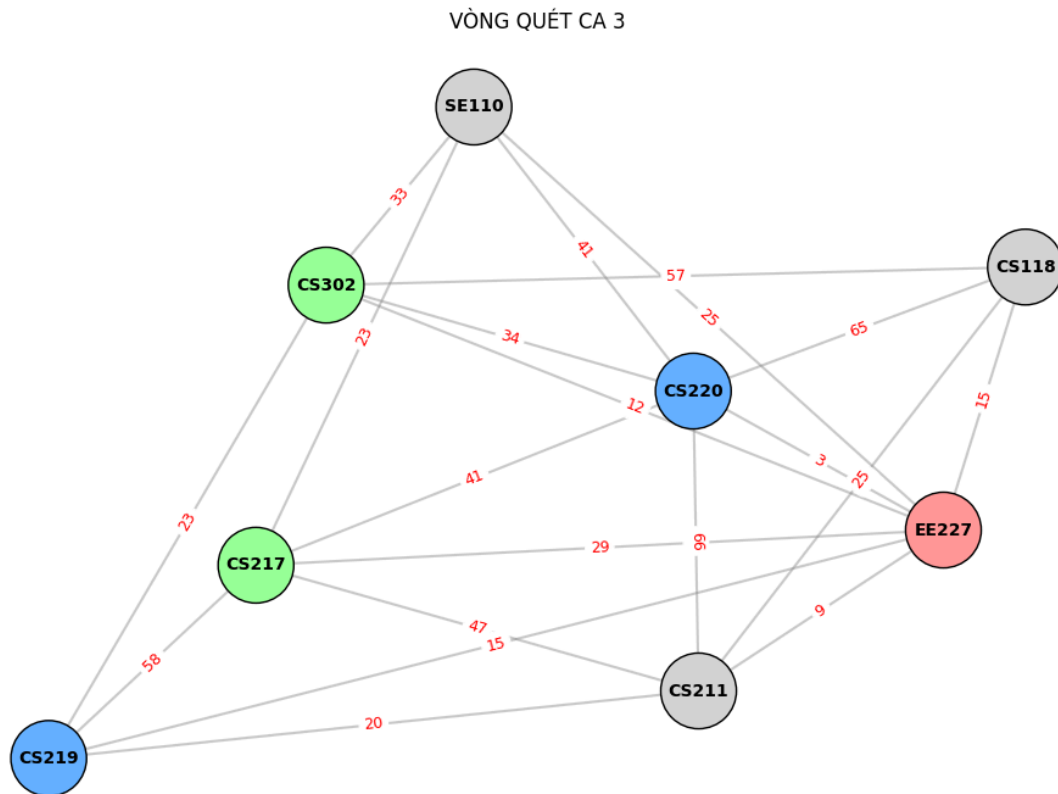


Tiếp theo là vòng quét thứ 2 cho ca thứ 2, là đỉnh có bậc 6 cho thành một màu, chỉ có CS219 là không xung đột nên sẽ cho ra 1 màu, giờ ta đã có 2 môn trong ca thi nên sẽ chuyển qua vòng quét tiếp theo.

VÒNG QUÉT CA 2

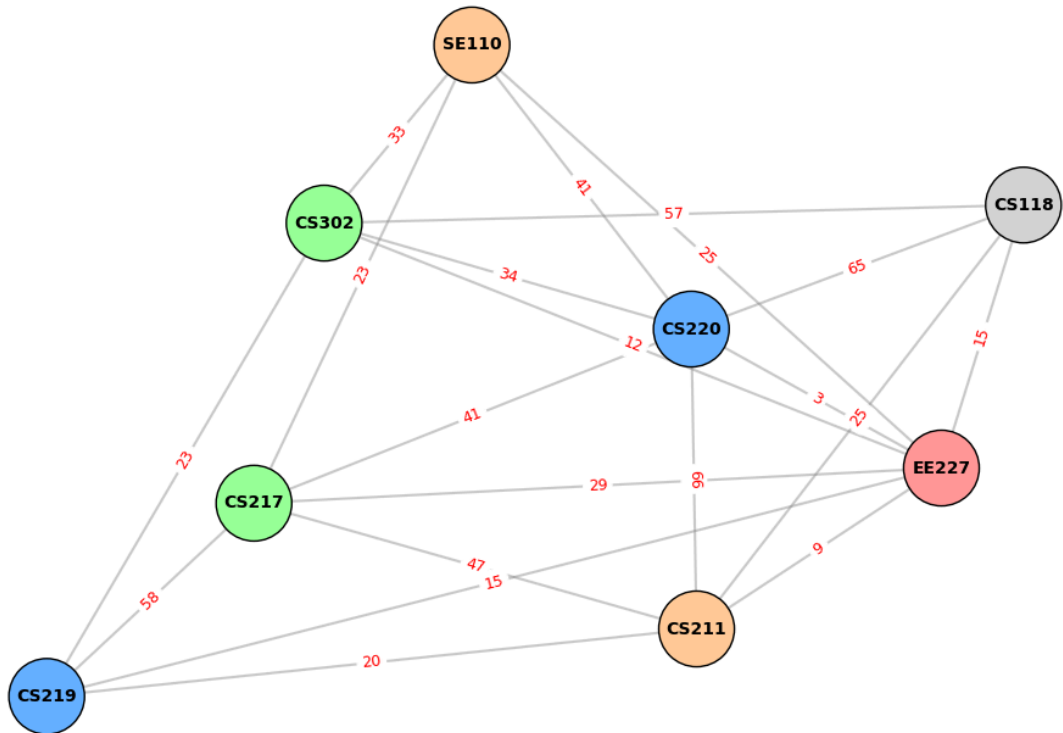


Tiếp theo là vòng quét thứ 3, cho môn CS217. Ở đây có 2 môn CS302 và CS118 là các đỉnh không xung đột, nhưng ta quét theo những môn đã được sắp xếp nên môn có bậc cao hơn sẽ vào ca này là CS302.



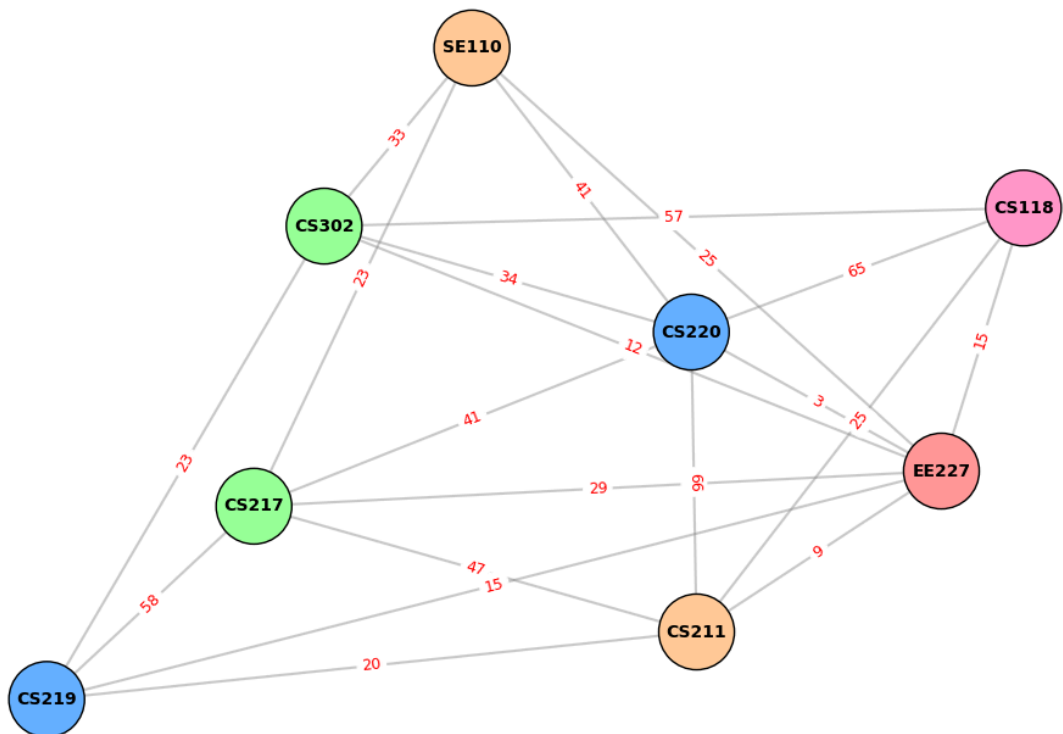
Tiếp vòng quét thứ 4, là CS211 với bậc 5. Tương tự ta sẽ tìm được SE110 phù hợp, và tô màu. Đây sẽ là ca thứ 4

VÒNG QUÉT CA 4



Và cuối cùng là môn thi còn lại, CS118 sẽ là ca cuối cùng - ca thứ 5

VÒNG QUÉT CA 5



## Kết quả

Thuật toán tô màu đồ thị sử dụng cho bài toán xếp lịch thi với số liệu đã cho sẽ cho ra 5 ca thi như sau:

```
--- TỔNG KẾT LỊCH THI ---  
Ca 1: EE227  
Ca 2: CS219, CS220  
Ca 3: CS217, CS302  
Ca 4: CS211, SE110  
Ca 5: CS118
```

Kết quả này thỏa mãn ràng buộc đề bài không có sinh viên nào bị trùng lịch thi.